DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING
Discover. Learn. Empower.

EXPERIMENT NO: 6

Student Name: Nakul Kumar **UID: 23BCS12354**

Branch: BE-CSE **Section/Group:** KRG-3A

Semester: 6th **Date of Performance:** 26/02/2026

Subject Name: System Design **Subject Code:** 23CSH-314

Aim

To design a highly scalable video streaming platform similar to YouTube that supports adaptive bitrate streaming (HLS/DASH), dynamic manifest handling, live streaming with sliding window playback, and real-time engagement features (likes and comments) with high availability and low latency.

Objectives

- Understand Adaptive Bitrate Streaming using HLS and DASH protocols.
- Analyze the role of manifest files in dynamic bandwidth handling.
- Design architecture for both Video-on-Demand (VOD) and Live Streaming.
- Implement scalable like and comment systems for static and live videos.
- Apply event-driven architecture for high-volume engagement processing.
- Ensure fault tolerance, high availability, and eventual consistency.

Procedure

1. Studied real-world streaming protocols including HLS and DASH.
2. Analyzed manifest file structure (.m3u8 and .mpd) and adaptive bitrate switching.
3. Designed live streaming architecture using sliding window manifest approach.
4. Implemented scalable like system using Redis counters and asynchronous persistence.
5. Designed comment service using NoSQL partitioning for large-scale videos.
6. Integrated WebSocket-based real-time chat system for live streaming.
7. Evaluated CDN-based global content distribution for low-latency playback.

Functional Requirements

1. Users should be able to upload videos.
2. Videos must be transcoded into multiple resolutions (240p–4K).
3. System should support Adaptive Bitrate Streaming (HLS/DASH).
4. Live streaming with near real-time playback should be supported.
5. Users should be able to like/unlike videos.
6. Users should be able to comment on videos.
7. Live streams should support real-time chat.
8. System should allow rewind functionality in live streams (DVR mode).

Core Entities of the System

• Users
• Videos
• VideoSegments
• ManifestFiles
• LiveStreams
• Interactions (Likes/Dislikes)
• Comments
• ChatMessages
• ViewHistory

API Design

1. Authentication API

- **Register**: POST /api/v1/auth/register
- **Login**: POST /api/v1/auth/login

2. Video & Feed API

- **Initialize Upload**: POST /api/v1/videos/upload/init (Returns S3 Pre-Signed URL)
- **Get Video Metadata**: GET /api/v1/videos/{video_id}
- **Get Personalized Feed**: GET /api/v1/feed (Pagination Supported)
- **Search Videos**: GET /api/v1/search?q={query}

3. Interaction API

- **Like Video**: POST /api/v1/videos/{video_id}/like
- **Add Comment**: POST /api/v1/videos/{video_id}/comments
- **Get Comments**: GET /api/v1/videos/{video_id}/comments

Non-Functional Requirements

1. System must handle extremely high throughput for concurrent uploads and millions of simultaneous views.
2. High Availability must be prioritized, ensuring no single point of failure.
3. System should follow an Eventual Consistency model for social metrics (view counts, likes) to optimize database write loads.
4. Video delivery latency should be minimized via Edge Caching and a Global CDN.
5. Transcoding pipeline must be horizontally scalable to handle viral upload spikes.

High Level Design (HLD)

Streaming Architecture (HLS & DASH)

The system supports:
• HLS (HTTP Live Streaming)
• DASH (Dynamic Adaptive Streaming over HTTP)

1 Video Upload & Processing

Flow:
Client → API Gateway → Upload Service → Object Storage

After upload:
Object Storage → Transcoding Service → Multi-resolution Segments

Each video is converted into:

240p
480p
720p
1080p

Each resolution is divided into small chunks (2–6 sec).

2 Manifest File Handling

HLS:
Master Manifest (.m3u8)
Variant Manifest (per quality)

DASH:
MPD File (.mpd)

Role of Manifest:

• Lists available qualities
• Lists segment URLs
• Enables adaptive bitrate switching

3 Adaptive Bitrate (ABR)

• Player measures bandwidth
• Selects suitable quality
• Downgrades on slow network
• Upgrades on stable network
• Switching occurs at segment boundaries

Live Streaming Architecture

Live streaming generates segments in real-time.

1 Live Segment Generation

Broadcaster → Ingestion Server → Real-Time Transcoder
Segments created continuously
(e.g., every 4 seconds)

2 Sliding Window Manifest

Manifest contains only recent segments
(e.g., last 5 minutes)
Old segments removed from manifest but may remain in storage.

Purpose:
• Keeps manifest small
• Reduces CDN load
• Improves playback efficiency
• Controls live latency

3 Manifest Refresh

Player refreshes manifest every few seconds
to detect new segments.

4 Live Rewind (DVR)

• Old segments stored in object storage
• Player calculates timestamp
• Fetches segment directly

Enables backward seeking.

Like System Design

Likes are high-frequency operations.

1️⃣ Static Video Likes

Flow:
Client → API → Like Service

Like Service:

- Redis counter increment
- Event pushed to Kafka
- DB updated asynchronously

Duplicate Prevention:

UNIQUE(user_id, video_id)
OR
Redis SETNX

2️⃣ Live Stream Likes

Live likes handled using:

Client → API → Redis INCR(stream_id)
Redis → Pub/Sub → WebSocket → Clients

Real-time updates pushed to viewers.
Database updated asynchronously.

Comment System Design

1️⃣ Static Video Comments

Stored in NoSQL DB:

Partition Key → video_id
Sort Key → timestamp

Advantages:
• Horizontally scalable
• Efficient pagination
• Fast retrieval

Replies stored using:
comment_id
parent_id
thread_root_id

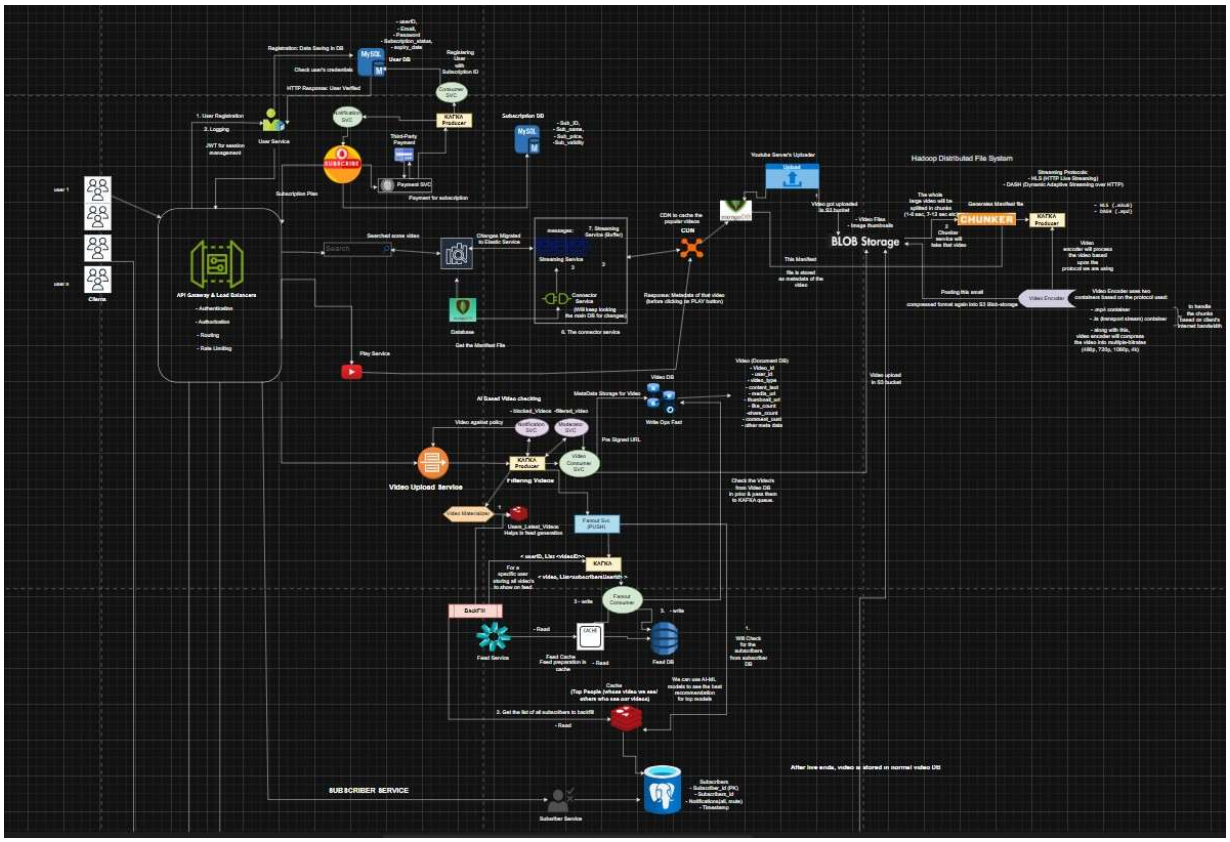
2 Live Stream Chat

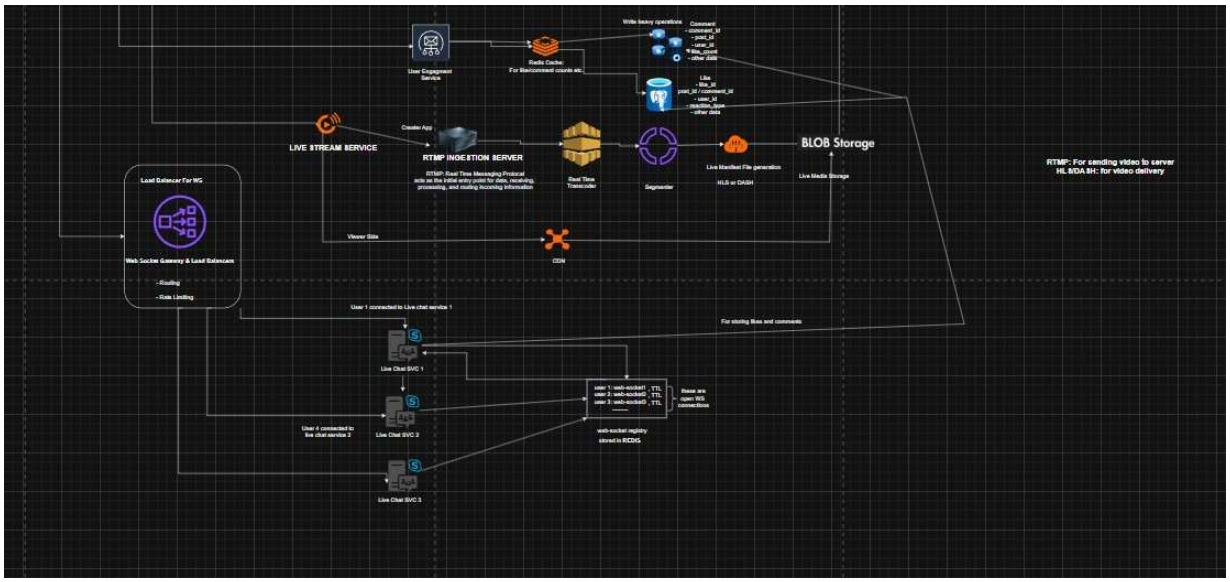
Uses WebSockets.

Flow:
Client → WebSocket Gateway → Chat Server
Chat Server → Pub/Sub → All viewers

Each stream has separate chat shard.

Persistence:
• Store limited live history
• Archive after stream ends


BLOB Storage
Hadoop Distributed File System
CHUNKER
API Gateway & Load Balancers
Video Upload Service
Video DB
Feed Service
Feed DB
SUBSCRIBER SERVICE


LIVE STREAM SERVICE
RTMP INGESTION SERVER
BLOB Storage
Load Balancer For WS
Web Socket Gateway & Load Balancer
RTMP: For sending video to server
HLS/DASH: for video delivery

Outcome

• Designed a scalable YouTube-like architecture supporting HLS and DASH.
• Implemented dynamic manifest refresh for live streaming.
• Designed sliding window mechanism for low-latency live playback.
• Built scalable like and comment systems for static and live videos.
• Applied event-driven architecture for engagement processing.
• Understood CDN-based global content delivery and adaptive bitrate streaming.